

An Algorithm for Cyclic Edge Connectivity of Cubic Graphs*

(Extended Abstract)

Zdeněk Dvořák, Jan Kára, Daniel Král', and Ondřej Pangrác

Department of Applied Mathematics and
Institute for Theoretical Computer Science**
Charles University

Malostranské náměstí 25, 118 00 Prague 1, Czech Republic.
{rakdver,kara,kral,pangrac}@kam.mff.cuni.cz.

Abstract. The cyclic edge connectivity is the size of a smallest edge cut in a graph such that at least two of the connected components contain cycles. We present an algorithm running in time $O(n^2 \log^2 n)$ for computing the cyclic edge connectivity of n -vertex cubic graphs.

1 Introduction

The cyclic connectivity as a graph parameter was introduced by Tait [14] already in 1880. A *cyclic edge cut* of a graph G is an edge cut such that at least two components of the graph without the cut contain a cycle. If G is not connected and at least two of its components contain a cycle, then an empty set of edges form a cyclic edge cut. The *cyclic edge connectivity* of a graph G is the size of a smallest cyclic edge cut. If G is connected, then each smallest cyclic edge cut splits G in exactly two components. A graph may have no cyclic edge cuts at all: K_4 , K_5 , $K_{3,3}$ and W_n are examples of such graphs.

The cyclic (vertex and edge) connectivity is an important graph parameter both from the theoretical and the practical points of view. As for the usual vertex and edge connectivity, the cyclic connectivity reflects the level of connectivity of a graph. However, both the usual vertex and edge connectivity are bounded for classes of graphs of bounded degree (say cubic graphs) and thus they do not say much about how the particular graph is connected. Hence, the cyclic connectivity can replace the usual connectivity in applications where the considered graphs have bounded maximum degree. Such applications range through all areas where graph algorithms have been applied and they include e.g. robustness of local computer networks and parallel computer architectures (networks are usually represented by graphs with bounded degrees since a single

* The REU programme where this research was originally started is supported by a cooperative research grant KONTAKT ME 521.

** Institute for Theoretical Computer Science (ITI) is supported by Ministry of Education of Czech Republic as project LN00A056.

computer/processor usually communicates only to a few other units) or the stability of tertiary structure of proteins. The cyclic connectivity actually fits the purpose of the latter application extremely well because a protein never unfolds (splits) to acyclic parts because of its primary and secondary structure [3] (with a possible exception for the exchange of small functional groups in biochemical reactions). From the theoretical point of view, not only the relation of the cyclic connectivity to other connectivity parameters [13,12] is interesting, but the cyclic edge connectivity also plays an essential role regarding the structure of cubic graphs [1,2,5,8,9,10]. The problem to develop a polynomial-time algorithm for the cyclic edge connectivity of cubic graphs was actually introduced to us by Roman Nedela in connection with designing an efficient algorithm for generating so-called snarks [11] which are cubic bridgeless 4-edge uncolorable graphs.

There is little of previous work on algorithms for the cyclic edge connectivity of either cubic graphs or graphs in general. The only published result is an $O(n^3 \log n)$ -algorithm by Lou et. al. [6] but the algorithm is incorrect [7]: the procedure `cyclic_connectivity` described on pages 251–253 in [6] does not always return the set of all minimal cyclic edge cuts as claimed in the paper. In addition, if case 3 applies, the number of cut sets may double and this is not considered in the time analysis. So, the algorithm does not always work in polynomial time and it does not even produce a correct result for some specific graphs. It is not clear whether their algorithm can be fixed [7].

In the present paper, we show two algorithms for computing the cyclic edge connectivity of cubic graphs, one running in time $O(n^3 \log n)$ and the other in time $O(n^2 \log^2 n)$. Both our algorithms are easy to implement and their time bounds do not involve any hidden large constants. So, both can be said to be practical and can be used for each of the purposes mentioned in the previous paragraph. The main idea of our algorithms is completely different from the idea of the algorithm of Lou et. al. whose algorithm recursively generates all cyclic edge cuts in a cubic graph. At the end of the paper, we briefly sketch how the algorithms can be extended to regular graphs and to a polynomial-time algorithm for the cyclic edge connectivity of arbitrary graphs.

First, we present our $O(n^3 \log n)$ -algorithm in Section 4. When presenting this algorithm, we explain ideas which are later used to design an algorithm with a better running time. The algorithm first computes the girth g of an input graph G . Recall that the *girth* of a graph G is the length of its shortest cycle. The girth is an upper bound on the cyclic edge connectivity of a cubic graph if $|V(G)| \geq 8$ (Lemma 1). Then, the algorithm computes minimum edge separations for all pairs of vertex disjoint full trees (see Section 2 for the definition) of depth at most $O(\log g)$ (the number of such pairs is $O(n^2 \log^2 \log n)$). The correctness of such an algorithm follows straightforwardly from Theorem 1.

Next, we present an $O(n^2 \log^2 n)$ -algorithm in Section 5. Again, the algorithm first computes the girth g of an input graph G . If the number of vertices of the input graph is small (at most 242), we run the $O(n^3 \log n)$ -algorithm for computing the edge connectivity. Otherwise, we find g edge-disjoint subgraphs

of G which are large in a sense introduced later in the paper. Since g is an upper bound on the cyclic edge connectivity, we want to test the existence of a cyclic edge cut of size at most $g - 1$. If such a cyclic edge cut exists, then one of the g subgraphs is disjoint from it. Hence it is enough to compute minimum edge separations between the g subgraphs and vertex disjoint full trees of depth at most $O(\log g)$. The number of such pairs is only $O(n \log n \log \log n)$ which reduces the time complexity compared to the algorithm from Section 4.

Both our algorithms assume that the input graph is cubic, connected, and contains at least 8 vertices because such cubic graphs always contain a cyclic edge cut (Lemma 1) which is not the case of cubic graphs on 2, 4 and 6 vertices, e.g., the triple edge, K_4 or $K_{3,3}$. This does not harm usability of our algorithms: There is only a single cubic graph on two vertices (a triple edge) and there are only three cubic graphs on four vertices (two triple edges, two double edges joined by a matching and a complete graph on four vertices). The only cubic graph without a cyclic edge cut on six vertices is $K_{3,3}$. These five exceptional cases can be easily checked in the very beginning of our algorithms.

2 Notation

In this section, we introduce notation and definitions used throughout the paper. Graphs considered in this paper may contain parallel edges but no loops. We write $V(G)$ and $E(G)$ for the vertex set and the edge set of a graph G , respectively. $G[W]$ denotes the subgraph of G induced by the vertex set W , $W \subseteq V(G)$. A graph G is said to be *cubic* if all its vertices have degree three.

An algorithmic procedure which is widely used in computer science that we use several times throughout the paper is a procedure for the *breadth-first search* (*BFS*) of a graph. You start at a vertex v_0 of G and assign sequentially the vertices of G labels which are equal to their distances from v_0 . We mean by a *BFS-graph* of depth d the graph induced by all the edges uv such that u is labelled by at most $d - 1$ and v by at most d . Note that a BFS-graph of depth d need not be a subgraph of G induced by the vertices of distance at most d from v_0 (the edges joining two vertices at the distance d are missing). If the BFS-graph is acyclic, we call it a *BFS-tree*. The vertex v_0 is said to be a *root* of the BFS-graph (BFS-tree). The vertices labelled with the number k form a *level* k of it. We sometimes abuse this notation a little and we root the BFS-graph at an edge. In such case, both the end-vertices of the edge are labelled by zero and the other vertices are labelled as described above.

We often use arguments related to cleverly chosen BFS-graphs to prove upper bounds on the girth of a graph. Observe the following simple facts: If the girth of G is at least $2k + 1$, then a BFS-graph of depth k rooted at any vertex of G is acyclic. If the girth of G is at least $2k + 2$, then the vertices of a BFS-graph of depth k rooted at any vertex of G induce an acyclic subgraph of G . This concept may actually be used to prove the following lemma from [6]:

Lemma 1. *Let G be a cubic graph of order n and girth g , $n \geq 8$. Then, $g \leq 2 \lceil \log_2(n/3 + 1) \rceil$. Moreover, G contains a cyclic edge cut of size g .*

Edge cuts are understood in this paper as decompositions of a vertex set of G into two sets. A cut is denoted by (A, B) where A and B are disjoint subsets of $V(G)$ such that $A \cup B = V(G)$, i.e., it is viewed as a partitioning of the vertex set to two subsets. The cut itself is formed by the edges joining a vertex of A and a vertex of B and its size is equal to the number of such edges. An edge cut is said to be *cyclic* if both $G[A]$ and $G[B]$ contain a cycle.

We consider a problem of finding cyclic edge cuts in cubic graphs. The definition introduced in this paragraph and the next lemma give us a tool to prove that an edge cut is a cyclic edge cut. Let G' be a connected subgraph of G . Then, the *activity* of G' is equal to $\sum_{v \in V(G')} (\deg_G v - \deg_{G'} v)$. Note that the activity of G' is at least the number of edges incident with the vertices of $V(G')$ but not included in G' . Note that the activity might be larger than this number because the edges not belonging to G' which join two vertices of G' are counted "twice". A motivation for this definition is obvious from the following lemma:

Lemma 2. *Let G_1 and G_2 be vertex-disjoint connected subgraphs of a cubic graph G , each of activity at least $k + 1$. Any edge cut separating G_1 and G_2 which is of size at most k is a cyclic edge cut.*

Proof. Let (A, B) be an edge cut of size at most k such that $V(G_1) \subseteq A$ and $V(G_2) \subseteq B$ and let E_{AB} be the edges joining A and B . We prove each of $G[A]$ and $G[B]$ contains a cycle. By symmetry, we focus on the case of $G[A]$ only. If the subgraph of G induced by $V(G_1)$ is not acyclic, then the claim is trivial. Assume that G_1 is an induced subtree of G .

Assume now for the sake of contradiction that $G[A]$ is a forest. Let E_1 be the set of edges incident with a vertex of G_1 but which are not in G_1 . Since G_1 is an induced subtree of G , the activity of G_1 is equal to $|E_1|$ and hence $|E_1| \geq k + 1$. Then, the number of leaves of the forest $G[A]$ which are not vertices of G_1 must be at least $|E_1 \setminus E_{AB}| = |E_1| - |E_1 \cap E_{AB}| \geq k + 1 - |E_1 \cap E_{AB}|$. Each such a leaf of $G[A]$ is incident with at least two edges of E_{AB} because G is cubic. Hence, the size of E_{AB} has to be at least $2(k + 1 - |E_1 \cap E_{AB}|) + |E_1 \cap E_{AB}| = 2k + 2 - |E_1 \cap E_{AB}| \geq k + 2$ which is impossible.

The graphs for which we often use Lemma 2 are trees. A *complete binary tree* of depth d is a tree rooted at a vertex v with levels $0, 1, \dots, d$ such that each vertex of levels $0, 1, \dots, d - 1$ has two children. For the sake of brevity, we call a complete binary tree of depth d just a *binary tree of depth d* . The number of vertices of the last level of a binary tree of depth d is 2^d and the activity of such a binary tree contained in a cubic graph is $1 + 2^{d+1}$. A *full tree* of depth d is a tree rooted at a vertex v with levels $0, 1, \dots, d$ such that the vertex v have three children and each vertex at a level between 1 and $d - 1$ has two children. The number of vertices of the last level is $3 \cdot 2^{d-1}$ and the activity of a full tree of depth d which is a subgraph of a cubic graph is $3 \cdot 2^d$.

3 Structural Results

Both our algorithms are based on a structural result from Theorem 1 presented in this section. The result seems to be of its own interest: Consider a cubic graph G with the cyclic edge connectivity κ . Then, G either contains a cycle of length κ or there exists a cyclic edge cut of size κ such that each of the components contains a full tree of depth $\Omega(\log \kappa)$. There is a short proof of the existence of a full tree of depth $\Omega(\log \kappa)$ in this setting, but we need that its activity is at least $\kappa + 1$. So an arbitrary function of order $\Omega(\log \kappa)$ does not suffice for our purposes. Even this stronger statement can be proved easily if κ is sufficiently large, but if we want to design practical algorithms, we need that the activity of such a tree is at least $\kappa + 1$ for all values of κ . It turned out that actually small values of κ (between 6 and 25) require quite a lot of work and the full proof of Theorem 1 is even a little technical in some aspects. Because of this and the space limitations, we only sketch the main ideas of its proof.

The following lemma was proved in [6], but we include its short proof for the sake of completeness:

Lemma 3. *Let G be a connected cubic graph with a cyclic edge connectivity κ and let (A, B) be a cyclic edge cut of size κ . Then, both $G[A]$ and $G[B]$ are connected graphs with minimum degree two. The number of degree-two vertices in each of $G[A]$ and $G[B]$ is exactly κ .*

Proof. $G[A]$ and $G[B]$ must clearly be connected graphs by the minimality of the edge cut. Assume that G contains a vertex v incident with two edges of a cyclic edge cut (A, B) and $v \in A$. Consider a cut (A', B') such that $A' = A \setminus \{v\}$ and $B' = B \cup \{v\}$. Both $G[A']$ and $G[B']$ contain a cycle (v has degree one in $G[A]$ and hence it is contained in no cycle of $G[A]$ and $B \subseteq B'$) but the size of the cut (A', B') is smaller than the size of the cut (A, B) because G is cubic. Thus, if (A, B) is a cyclic edge cut of size κ , no vertex is incident with two edges of the cut. The lemma now follows straightforwardly.

Lemma 4. *Let H be a connected graph with n_2 vertices of degree two and n_3 vertices of degree three. If H does not contain a full tree of depth d , $d \geq 1$, and $n_3 > (2^d - 2)n_2$, then H contains a cycle of length at most $2d$.*

Proof. Let us consider a vertex v of degree three in H . If the BFS-graph of depth d rooted at v is not acyclic, then H contains a cycle of length at most $2d$. Otherwise, the BFS-graph of depth d rooted at v is a tree, but it cannot be a full tree. Hence, there is a non-leaf vertex of degree two contained in it. Hence the distance between v and the nearest degree-two vertex is at most $d - 1$. Since the choice of v was arbitrary, we may conclude that each vertex of degree three is at distance at most $d - 1$ from a vertex of degree two. The number of neighbors at distance at most $d - 1$ from a vertex of degree two is at most $2^d - 2$ for a fixed degree two vertex. However, this implies that $n_3 \leq (2^d - 2)n_2$.

Theorem 1. *Let G be a cubic graph with a cyclic edge connectivity κ , $\kappa \geq 1$. Then at least one of the following holds:*

1. G contains a cycle of length κ .
2. G contains a cyclic edge cut of size κ such that each of the two parts contains a full tree of depth $d = \lceil \log_2 \frac{\kappa+1}{3} \rceil$.

Proof. Assume that G does not contain a cycle of length κ , i.e., the girth of G is at least $\kappa+1$. If none of the cyclic edge cuts (A, B) of size κ has the property that both $G[A]$ and $G[B]$ contain the full tree, then consider one where the number of vertices of A is as small as possible. Let $H = G[A]$. Clearly, $H \neq C_\kappa$ and the girth of H (a subgraph of G) is at least $\kappa+1$. By Lemma 3, H is connected, its minimum degree is two and the number of degree-two vertices of H is exactly κ . Let $n_2 = \kappa$ be the number of vertices of degree two (in the rest, we use both n_2 and κ for this same number depending on which of the two quantities we like to emphasize) and n_3 the number of vertices of degree three of H .

We prove the theorem only for $\kappa \geq 26$ (hence $d \geq 4$). A (more or less technical) proof of this theorem for smaller values of κ is left due to space limitations (it is necessary to use finer bounds on the number of vertices in BFS-trees). We bound the number of vertices of H . If H does not contain a full tree of depth d , then $n_3 \leq (2^d - 2)n_2$ by Lemma 4. We bound the number of vertices of H as follows:

$$n_2 + n_3 \leq (2^d - 1)\kappa \leq \left(2^{\log_2 \frac{\kappa+1}{3} + 1} - 1\right) \kappa \leq \left(2 \cdot \frac{\kappa+1}{3} - 1\right) \kappa \leq \frac{2\kappa^2}{3}$$

Let $l = \lfloor \kappa/2 \rfloor$. Since the girth of H is at least $\kappa+1$, the BFS-graph of depth l rooted at any vertex of degree three is acyclic. It can be shown that H contains no two adjacent vertices of degree two and thus the number of vertices at level i , $1 \leq i \leq l$, of this BFS-tree is at least $3 \cdot 2^{\lfloor \frac{i-1}{2} \rfloor}$. Hence the BFS-tree contains at least the following number of vertices:

$$\begin{aligned} & 1 + 3 \cdot 2^0 + 3 \cdot 2^0 + 3 \cdot 2^1 + 3 \cdot 2^1 + 3 \cdot 2^2 + \dots + 3 \cdot 2^{\lfloor \frac{l-1}{2} \rfloor} = \\ & 1 + 3 \cdot 2^0 + 3 \cdot 2^1 + \dots + 3 \cdot 2^{\lfloor \frac{l-2}{2} \rfloor} + 3 \cdot 2^0 + 3 \cdot 2^1 + \dots + 3 \cdot 2^{\lfloor \frac{l-1}{2} \rfloor} = \\ & 1 + 3 \left(2^{\lfloor l/2 \rfloor} - 1\right) + 3 \left(2^{\lfloor l/2 \rfloor} - 1\right) = 3 \cdot 2^{\lfloor l/2 \rfloor} + 3 \cdot 2^{\lfloor l/2 \rfloor} - 5 \end{aligned}$$

Since l is integer, we further bound this number of vertices as follows:

$$3 \cdot 2^{\lfloor l/2 \rfloor} + 3 \cdot 2^{\lfloor l/2 \rfloor} - 5 \geq 3 \cdot 2 \cdot 2^{l/2} - 5 = 6 \cdot 2^{l/2} - 5$$

We get using the inequality $l = \lfloor \kappa/2 \rfloor \geq \kappa/2 - 1/2$ the following:

$$6 \cdot 2^{l/2} - 5 \geq 6 \cdot 2^{(\kappa-1)/4} - 5$$

We may conclude that H must contain at least $6 \cdot 2^{(\kappa-1)/4} - 5$ vertices. But H contains at most $2\kappa^2/3$ vertices as proved above. This is impossible because $2\kappa^2/3 < 6 \cdot 2^{(\kappa-1)/4} - 5$ for $\kappa \geq 26$.

4 An $O(n^3 \log n)$ -Algorithm

We are now ready to present our first algorithm for the cyclic edge-connectivity of cubic graphs:

Theorem 2. *There is an algorithm for computing the cyclic edge connectivity of cubic graphs running in time $O(n^3 \log n)$.*

Proof. We assume that the number of vertices of the input graph G is at least 8. By Lemma 1, such a graph G always contains a cyclic edge cut. The cases when G has 2, 4 or 6 vertices can be easily handled separately as explained at the end of Section 1. A pseudocode of the algorithm is the following (the subroutines `findpaths` and `findcut` are described in detail at the end of the proof):

Input: a cubic graph G of order at least 8
Output: a cyclic edge cut of the minimum size

```

cutsizesize := girth( $G$ )
cut := edges incident with a cycle of length cutsizesize
forall  $v \in V(G)$  do
  forall  $w \in V(G)$  do
    d := -1
    paths :=  $\emptyset$ 
    repeat
      d := d + 1
       $T_v$  := a full tree of depth d rooted at v
       $T_w$  := a full tree of depth d rooted at w
      if  $T_v$  and  $T_w$  are not vertex-disjoint then break
      paths := findpaths( $T_v$ ,  $T_w$ , paths)
      if  $|\text{paths}| < 3 * 2^d$  and  $|\text{paths}| < \text{cutsizesize}$  then
        cutsizesize =  $|\text{paths}|$ 
        cut = findcut( $T_v$ ,  $T_w$ , paths)
      fi
    while  $3 * 2^d < \text{cutsizesize}$ 
  endfor
endfor
output cutsizesize and cut

```

The algorithm first computes the girth g of the input graph G . This can be straightforwardly done in time $O(n^2)$ by running a BFS routine from each of the vertices of G . The girth g which is of order $O(\log n)$ (Lemma 1) is an upper bound on the cyclic edge connectivity of G . The algorithm maintains the size k of the smallest cyclic edge cut found so far (this value is initially set to the girth of G). In the main cycle, the minimum edge separations between the full trees of depth d rooted at v and at w (if such two trees are vertex-disjoint) are computed for all pairs v and w of the vertices of G and all the values of $0 \leq d \leq \lceil \log_2 k/3 \rceil$. If the size of the edge cut is smaller than $3 \cdot 2^d$, then the edge cut is cyclic by

Lemma 2. Hence, we can use this edge cut as a new candidate for a minimum cyclic edge cut (if it is smaller than the smallest one found so far).

We use a simple flow algorithm to find the edge cut for each pair of full trees which either finds $3 \cdot 2^d$ edge-disjoint paths between the full trees or the edge cut of size smaller than $3 \cdot 2^d$. The algorithm at each iteration either augments the flow (increase the number of edge-disjoint paths) between the trees or finds an edge cut of the size equal to the number of the paths (see also the comments at the end of the proof). The number of iterations is bounded by $3 \cdot 2^d$. Each of the iterations requires time which is linear in the number of edges of G , i.e., $O(n)$. For a fixed pair of vertices, we have to run at most $3 \cdot 2^0 + 3 \cdot 2^1 + 3 \cdot 2^2 + \dots + 3 \cdot 2^{\lceil \log_2 g/3 \rceil}$ iterations, hence the number of iterations of the flow algorithm for a fixed pair of vertices is at most $O(g)$ for all the pairs of full trees of depths $1, \dots, O(\log g)$ altogether. Since moreover each call of the flow algorithm needs time $O(n)$, as we discuss below, the algorithm runs in time $O(n^3 g) = O(n^3 \log n)$. We may further improve a running time of the algorithm by using the edge-disjoint paths found between the full trees of depth d rooted at v and w as a starting set of paths between the full trees of depth $d + 1$ rooted at v and w .

The correctness of the algorithm follows from Theorem 1. Let κ be the cyclic edge connectivity of G . If $\kappa = 0$, then G is disconnected and the algorithm clearly works correctly. If there is a cycle of length κ in G , then the minimum cyclic edge cut is equal to the girth of G and we find it in the very beginning of the algorithm. Otherwise, there exists an edge cut (A, B) of size κ such that both $G[A]$ and $G[B]$ contain full trees of depth $\lceil \log_2 \frac{\kappa+1}{3} \rceil$ due to Theorem 1. At a certain step of the algorithm, the minimum edge separation between these two full trees was computed and its size was at most κ (the cyclic edge cut is one of edge cuts between A and B). Since the activity of a full tree of depth $\lceil \log_2 \frac{\kappa+1}{3} \rceil$ is at least $\kappa + 1$, we found a cyclic edge cut of size κ (any cut of size κ between these full trees is cyclic by Lemma 2).

We give some comments to the implementation. The algorithm uses the following subroutines: **findpaths**(A, B, S) and **findcut**(A, B, S). Both the subroutines can be based on any standard flow algorithm [4] for edge-disjoint paths which in each iteration constructs an auxiliary graph and either enlarge the set of edge-disjoint paths or finds an edge cut corresponding to the paths. Each such iteration runs in time $O(n)$ because G is cubic and hence its number of edges is $O(n)$. The subroutine **findpaths**(A, B, S) finds the largest number of edge-disjoint paths between the vertex disjoint subgraphs A and B provided a set S of some edge-disjoint paths between the subgraphs A and B (the flow algorithm is applied to the graph G with each of A and B contracted to a single vertex). The running time of this subroutine is $O((k + 1 - k_0)n)$ where k is the number of edge-disjoint paths between A and B , k_0 is the number of paths of S and n is the number of vertices of G . This is because there have to be performed $k + 1 - k_0$ iterations of the flow algorithm. The subroutine **findcut**(A, B, S) finds an edge cut between the vertex disjoint subgraphs A and B if S is a set of the largest number of edge-disjoint paths between A and B . The running time of this subroutine is bounded by $O(n)$.

5 An $O(n^2 \log^2 n)$ -Algorithm

We first state that if the girth or the order of a cubic graph is sufficiently large, then one can construct a large number of edge-disjoint subgraphs of a certain type in an algorithmic way:

Lemma 5. *Let G be a cubic graph of order at least 243 and girth at least five. Then G contains 12 edge-disjoint full trees of depth two. Moreover, 12 such trees can be found in time $O(n)$ where n is the number of vertices of G .*

Proof. Since the girth of G is at least five, the BFS-graph of depth two rooted at any vertex of G must be a full tree (of depth two). We find 12 vertices of G such that the distance between any pair of them is at least four. The BFS-trees of depth two rooted at such vertices are edge-disjoint and as argued above they are actually full trees of depth two.

Take an arbitrary vertex of G and mark this vertex together with all the vertices at distance at most three from it. Then take an unmarked vertex, mark it and mark also all the vertices at distances at most three from it. At each step, at most $1 + 3 + 6 + 12 = 22$ vertices get marked. Since G has at least $243 > 22 \cdot 11$ vertices, we definitely find at least 12 such vertices. The just described greedy algorithm can be easily implemented in time $O(n)$.

Lemma 6. *Let G be a cubic graph of girth at least g , $g \geq 13$. Then G contains at least g edge-disjoint binary trees of depth $\lceil \log_2 \frac{g-1}{2} \rceil$. Moreover, g such trees can be found in time $O(n)$ where n is the number of vertices of G .*

Proof. Let $d = \lceil \log_2 \frac{g-1}{2} \rceil$ and $D = \lfloor \frac{g-1}{2} \rfloor$. Consider any edge e of the graph G . Consider the BFS-graph rooted at e of the depth D . Due to the girth assumption, the vertices of the levels $0, \dots, D-1$ induce a tree in G . Consider the following binary trees: Let $l = \lfloor D/d \rfloor$. Take two binary trees of the depth d rooted at the two vertices of the level 0; these trees contain vertices of levels $0, \dots, d$. Take 2^{d+1} binary trees of the depth d rooted at 2^{d+1} vertices of the level d ; these trees contain vertices of levels $d, \dots, 2d$. Proceed in this manner upto the trees rooted at vertices of the level $d(l-1)$. All the constructed trees are edge-disjoint. Their number is equal to $2 + 2^{d+1} + \dots + 2^{d(l-1)+1} = 2^{\frac{dl-1}{2}-1}$.

We check that $2^{\frac{2^{dl}-1}{2}-1} \geq g$ if $g \geq 13$. If $13 \leq g \leq 17$, then $d = 3$, $l = 2$ and hence the number of the trees is $2^{\frac{6-1}{2}-1} = 18$. If $18 \leq g \leq 24$, then $d = 4$, $l = 2$ and hence the number of the trees is $2^{\frac{8-1}{2}-1} = 34$. Finally, if $25 \leq g \leq 26$, then $d = 4$, $l = 3$ and hence the number of the trees is $2^{\frac{12-1}{2}-1} = 546$. We proceed as follows for the remaining values of g :

$$\begin{aligned}
 2 \cdot \frac{2^{dl} - 1}{2^d - 1} &\geq 2 \cdot \frac{2^{(D/d-1)d} - 1}{g - 1 - 1} = 2 \cdot \frac{2^{D-d} - 1}{g - 2} \geq 2 \cdot \frac{2^{D+1}/(g+1) - 1}{g - 2} = \\
 &2 \cdot \frac{2^{D+1} - g - 1}{g^2 - g - 2} \geq 2 \cdot \frac{2^{g/2} - g - 1}{g^2 - g - 2}
 \end{aligned}$$

The obtained expression is greater than g if $g \geq 27$.

The trees can be clearly found algorithmically in time $O(n)$. Simply take any edge e of G , run the BFS routine and output g binary trees of depth d constructed in the above way.

We are now ready to present our algorithm running in time $O(n^2 \log^2 n)$:

Theorem 3. *There is an algorithm for computing the cyclic edge connectivity of cubic graphs running in time $O(n^2 \log^2 n)$.*

Proof. Let us briefly describe the algorithm: If the number of vertices of the input cubic graph G is smaller than 243, we run the algorithm from Theorem 2. In the other case, we start by computing the girth g of G . This can be straightforwardly done in time $O(n^2)$ by running a BFS routine from each of the vertices of G . We create a set A_0 of g edge-disjoint subgraphs of G :

- If $2 \leq g \leq 4$, then A_0 consists of any g different edges of G .
- If $5 \leq g \leq 12$, then A_0 consists of g edge-disjoint full trees of depth 2. Such a set A_0 can be constructed in time $O(n)$ due to Lemma 5.
- If $13 \leq g$, then A_0 consists of g edge-disjoint binary trees of depth $\lceil \log_2 \frac{g-1}{2} \rceil$. Such a set A_0 can be constructed in time $O(n)$ due to Lemma 6.

Note that the activity of any subgraph contained in A_0 is at least g .

Our algorithm computes a minimum edge separation for all pairs of a subgraph $A \in A_0$ and a full tree rooted at $v \in V(G)$ of depth d for $0 \leq d \leq \lceil \log_2 k/3 \rceil$ for k which is the size of the cyclic edge cut found so far (initially $k = g$). As in the algorithm from Theorem 2, we use a simple flow algorithm to compute edge-disjoint paths between a full tree and a subgraph A . We also use the paths between the full tree of depth d and A as an initial set of paths between the full tree of depth $d+1$ and A . If the found edge cut is smaller than $3 \cdot 2^d$ (the activity of the full tree of depth d) and than g , the edge cut is cyclic (by Lemma 2) and if it is smaller than the cyclic edge cut found so far, we have a new upper bound on the cyclic edge cut.

The number of iterations for a fixed pair of a subgraph A and a vertex v is $O(g) = O(\log n)$ for all the full trees rooted at v together. Each iteration takes time $O(n)$. The number of subgraphs in A_0 is $g = O(\log n)$ and thus the running time of the whole algorithm is $O(n^2 \log^2 n)$.

We prove the correctness of our algorithm: Let κ be the size of the smallest cyclic edge cut of G . If $\kappa = 0$, then G is disconnected and the algorithm clearly finds the empty cyclic edge cut. If $\kappa = g$, then the cyclic edge cut of size g is found in the first phase of the algorithm. If $\kappa < g$, then there is a cyclic edge cut (B, C) such that both $G[B]$ and $G[C]$ contain full trees of depth $\lceil \log_2 \frac{\kappa+1}{3} \rceil$ due to Theorem 1. Besides this, one of the graphs $G[B]$ and $G[C]$ contains a subgraph $A \in A_0$ because one of the subgraphs of A_0 does not contain an edge of the cut (the subgraphs of A_0 are edge-disjoint, their number is g and the size of the cyclic edge cut is $\kappa < g$). Assume that $G[B]$ does. Now, $G[B]$ contains a subgraph A from the set A_0 and $G[C]$ contains a full tree of depth $\lceil \log_2 \frac{\kappa+1}{3} \rceil$.

At the step when we consider the pair consisting of the subgraph A and a full tree of depth $\lceil \log_2 \frac{\kappa+1}{3} \rceil$ in $G[C]$, we found a cyclic edge cut of size κ (we found an edge cut of size at most κ and such an edge cut is cyclic by Lemma 2).

6 Future Research

In this section, we briefly discuss two directions for possible future research. We have already obtained in both these directions partial results, but we keep working on improving both of the obtained results.

Graphs of the Bounded Degree

Our algorithms can be extended to the case of graphs with the maximum vertex degree D preserving the running times $O(n^3 \log n)$ and $O(n^2 \log^2 n)$. However, in this case, the running times involve huge multiplicative constants which make both the algorithms impractical. We sketch the main ideas: First, remove vertices of degree zero and one and suppress vertices of degree two (this does not change the cyclic edge connectivity). Since the minimum degree of a graph is at least three now, its girth g is at most $O(\log n)$ and its cyclic edge connectivity is at most Dg . Consider a cyclic edge cut (A, B) of size k . Let us call *cut vertices* the vertices incident with the edges of the cut. Either A contains at most $k \cdot D^{\log_2 k} = k^{1+\log_2 D}$ vertices or it contains a vertex whose distance from each cut-vertex is at least $\log_2 k$ (the proof of this fact proceeds in a way analogous to the proof of Lemma 4). As in Theorem 1, if $k \leq Dg$, we may conclude in the former of the two cases that the girth of A is at most $O(\log k^{1+\log_2 D}) = O(\log D \log Dg)$ which is impossible unless $g \leq O(\log^2 D)$.

Based on the discussion of the previous paragraph, we may conclude: Either the cyclic edge connectivity of an input graph is bounded by $O(\log^2 D)$ or there is a smallest cyclic edge cut of size k such that each part contains a BFS-tree of depth at least $\log_2 k$ (this is an analogue of a full tree). Hence, it is enough to try to separate BFS-trees of depth at most $O(\log_2(Dg))$ and subgraphs of activity at most $O(\log^2 D)$. The number of such pairs is at most $O(n^2)$ where the hidden multiplicative constant is of order $D^{O(\log^2 D)}$. The main goal of Theorem 1 is to reduce the number of subgraphs of bounded activity which are necessary to check to one, namely to a cycle which actually makes both our algorithms for cubic graphs practical.

Of course, if the order of the input graph is large enough, it is possible to find a sufficient number of large edge-disjoint subgraphs similarly as in Lemmas 5 and 6 and decrease the number of pairs to separate to $O(n \log n)$ which gives an algorithm running in time $O(n^2 \log^2 n)$.

Arbitrary Graphs

We sketch an $O(n^{11})$ -time algorithm for arbitrary graphs: A modified BFS-routine is used to find a cycle with the smallest activity (let α be this activity). If

the activity of such a cycle is not an upper bound on the cyclic edge connectivity, the input graph is one of the several special types which are handled separately. In the general case, the algorithm ranges through all paths with activity approximately α (there are at most $O(n^8)$ pairs of such paths) and it finds a separation for each such pair. The smallest edge separation is the sought cyclic edge cut. The running time can be further improved by showing that it is enough to range only through pairs of a certain special type as pointed out to us by Jiří Sgall.

Acknowledgement. This research was started when the authors took part in the DIMACS/DIMATIA Research Experience for Undergraduates programme in 2001 under the supervision of Jeff Kahn from Rutgers University. This programme is a joint project of DIMACS at Rutgers University and DIMATIA at Charles University. We temporarily stopped working on this problem when the programme had been finished and we restarted our work at the end of the year 2002.

The authors thank all the anonymous referees for their comments which helped to improve the style of this paper and the clarity of our arguments.

References

1. R. E. L. Aldred, D. A. Holton, B. Jackson: Uniform cyclic edge connectivity in cubic graphs, *Combinatorica* **11** (1991), 81–96.
2. L. D. Andersen, H. Fleischner, B. Jackson: Removable edges in cyclically 4-edge-connected cubic graphs, *Graphs Comb.* **4**(1) (1988), 1–21.
3. R. L. Caret, K. J. Dennistion, J. J. Topping: Principles and applications of inorganic, organic & biological chemistry, Wm. C. Brown Publishers, Boston (1997).
4. T. H. Cormen, C. E. Leiserson, R. L. Rivest: Introduction to Algorithms, MIT Press, Boston (1990).
5. H. Fleischner, B. Jackson: A note concerning some conjectures on cyclically 4-edge connected 3-regular graphs, *Ann. Disc. Math.* **41** (1989), 171–177.
6. D. Lou, L. Teng, X. Wu: A polynomial algorithm for cyclic edge connectivity of cubic graphs, *Australasian J. Comb.* **24** (2001), 247–259.
7. D. Lou: private communication.
8. W. McCuaig: Cycles through edges in cyclically k -connected cubic graphs, *Discrete Math.* **103** (1) (1992), 95–98.
9. W. McCuaig: Edge reductions in cyclically k -connected cubic graphs, *J. Comb. Theory Ser. B* **56** (1) (1992), 16–44.
10. R. Nedela, M. Škoviera: Atoms of cyclic connectivity in cubic graphs, *Math. Slovaca* **45** (1995), 481–499.
11. R. Nedela, M. Škoviera: Decompositions and reductions of snarks, *J. Graph Theory* **22** (1996), 253–279.
12. B. Peroche: On Several Sorts of Connectivity, *Discrete Mathematics* **46** (1983), 267–277.
13. M. D. Plummer: On the cyclic connectivity of planar graphs, in: *Graph Theory and Applications*, Springer-Verlag, Berlin (1972), 235–242.
14. P. G. Tait, Remarks on the colouring of maps, *Proc. Roy. Soc. Edinburg* **10** (1880), 501–503.